CHAPTER 4

System Design

4 Chapter (4) : System Design

4.1 Introduction

- The development of an Autonomous Emergency Braking (AEB) system requires a seamless synergy between high-fidelity environment simulation and standardized automotive software architectures. This chapter details the comprehensive design and structural framework of our AEB integration project, which bridges the gap between virtual physics and embedded control logic. By utilizing the CARLA Simulator for realistic environmental modeling and Vector SIL Kit as a high-performance communication middleware, we have established a robust Software-in-the-Loop (SIL) environment.
- The primary objective of this architecture is to implement a safety-critical algorithm within an AUTOSAR-compliant Software Component (SWC). This component is designed to process dynamic sensor data specifically raw LiDAR measurements and ego-vehicle speed to perform real-time risk assessment. Central to this design is the calculation of Time-to-Collision (TTC) and the management of multi-level Forward Collision Warnings (FCW) that trigger automated braking responses.
- Throughout this chapter, we explore the system through multiple design lenses. We begin with the High-Level System Architecture, illustrating the data flow between the simulator and the Virtual ECU (VECU). We then examine the Logical Component Breakdown, which partitions the system into perception, decision, and actuation layers. This is followed by a detailed mapping of the AUTOSAR Run-Time Environment (RTE) and UML diagrams including Class, Sequence, and Activity diagrams that define the static and dynamic behavior of the software. Finally, we discuss the Technology Stack and User Interface, specifically focusing on the diagnostic dashboards used to monitor vehicle parameters like RPM and speed.
- Together, these elements provide a holistic blueprint for a modern, scalable, and verifiable AEB system.
- To further strengthen the validity of the proposed system, this chapter also emphasizes the importance of synchronization and data consistency across all integrated modules. Special attention is given to timing constraints, message latency, and the deterministic behavior required for safety-critical applications. By ensuring reliable communication between the simulation environment and the AUTOSAR-based control logic, the system can maintain accurate and timely decision-making under varying conditions. This not only enhances the credibility of the SIL setup but also demonstrates its potential as a foundation for future transition toward Hardware-in-the-Loop (HIL) testing and real-world deployment.

4.2 High-Level System Architecture

- The Autonomous Emergency Braking (AEB) system is designed using a distributed architecture that integrates high-fidelity simulation with automotive-grade software standards. The overall system consists of three primary components: the CARLA Simulator, the SIL Kit middleware, and the AUTOSAR-based Software Component (SWC), each responsible for a distinct role within the data processing pipeline.
- The system follows the AUTOSAR Classic Platform layered architecture, which ensures modularity, scalability, and hardware abstraction. This architecture separates application-level logic from low-level hardware dependencies through the use of the Run-Time Environment (RTE). The layered structure includes, from bottom to top, the Microcontroller Abstraction Layer (MCAL), ECU Abstraction Layer, Services Layer, RTE, and the Application Layer where the AEB algorithm resides as an SWC.
- The CARLA Simulator acts as the environment generator, providing a realistic virtual representation of vehicle dynamics, surrounding traffic, and sensor outputs. In this project, it generates raw LiDAR data and vehicle state information such as speed, which are essential inputs for the AEB algorithm.
- The SIL Kit middleware serves as the communication backbone of the system, enabling seamless data exchange between simulation and control components. Communication is implemented using Controller Area Network (CAN) messages, ensuring compatibility with automotive standards and enabling deterministic data transfer.
- On the receiving end, the AUTOSAR SWC processes incoming sensor data to perform real-time risk assessment. Based on the computed Time-to-Collision (TTC) and predefined thresholds, the system generates appropriate Forward Collision Warnings (FCW) and braking commands.
- Additionally, monitoring and visualization tools play a crucial role in system validation. The SIL Kit dashboard and CANoe environment provide real-time insights into CAN signals, allowing observation of key vehicle parameters such as speed, RPM, and braking status during simulation.

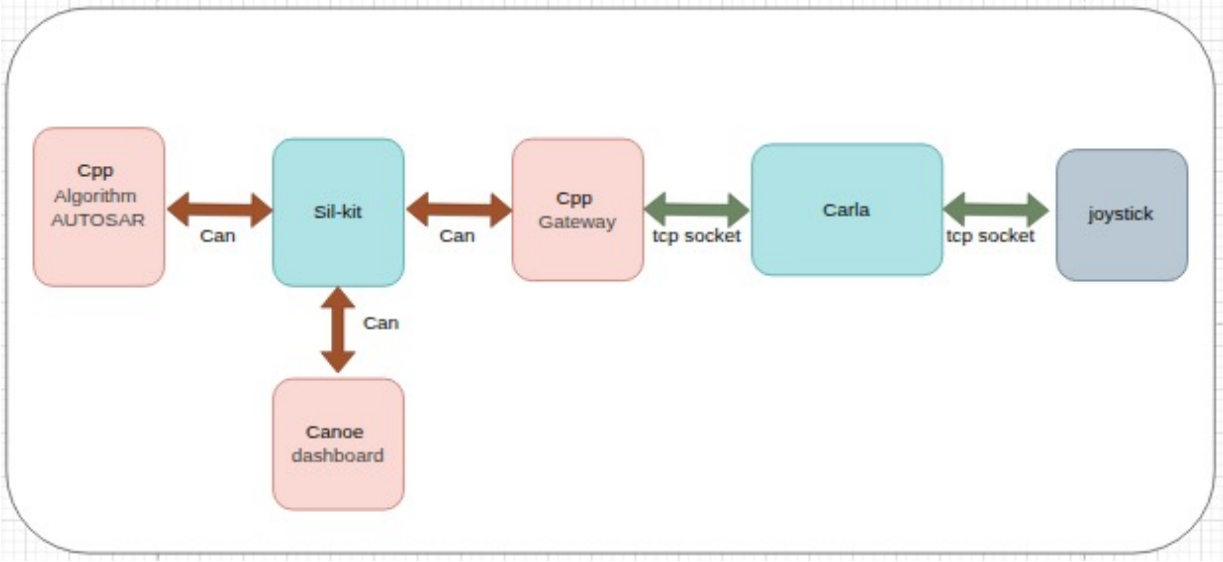


Figure 4.1: High-level System Architecture Diagram

The high-level system architecture is illustrated in Figure 4.1. The diagram highlights the interaction between the main system components and the communication channels used for data exchange. The AUTOSAR-based AEB algorithm communicates with the SIL Kit via CAN messages, which are also visualized through the CANoe dashboard for monitoring purposes. A gateway module implemented in C++ bridges the communication between the SIL environment and the CARLA simulator using TCP sockets. Additionally, a joystick interface is connected to CARLA to provide manual control inputs during simulation scenarios. This architecture ensures a clear separation between simulation, communication, and control logic, enabling modular development and efficient testing.

4.3 CARLA Simulation Environment

The CARLA simulator is utilized as the primary environment for testing and validating the Autonomous Emergency Braking (AEB) system. It provides a high-fidelity virtual platform capable of simulating realistic vehicle dynamics, road conditions, and interactions with surrounding objects such as vehicles, pedestrians, and static obstacles. In this project, the simulation operates in synchronous mode with a fixed time step of 0.05 seconds. This ensures deterministic execution, where all components of the system update in a controlled and consistent manner. Such behavior is essential for safety-critical systems, as it guarantees reliable timing for sensor processing and decision-making.

The simulation environment includes an ego vehicle equipped with sensors, as well as multiple dynamic and static actors. These include lead vehicles, crossing pedestrians, bicycles, and roadside obstacles. Each actor is programmed with predefined behaviors to create controlled test scenarios that challenge the AEB system under different conditions

Figure 4.2. CARLA simulator

4.3.1 Sensor Modeling and Perception (LiDAR)

LiDAR Sensors:

In the CARLA environment, LiDAR (Light Detection and Ranging) sensors utilize rotating raycasting to generate a high-fidelity 3D representation of the surroundings. Similar to real-world sensors, CARLA allows for precise configuration of the number of channels (vertical resolution), points per second, and rotation frequency. By measuring the "Time of Flight" of laser pulses, the sensor provides a dense point cloud that includes the exact 3D coordinates (x, y, z) of every hit point relative to the ego vehicle. This allows the AEB system to perceive the precise shape, volume, and distance of static and dynamic obstacles with much higher spatial resolution than a standard radar.

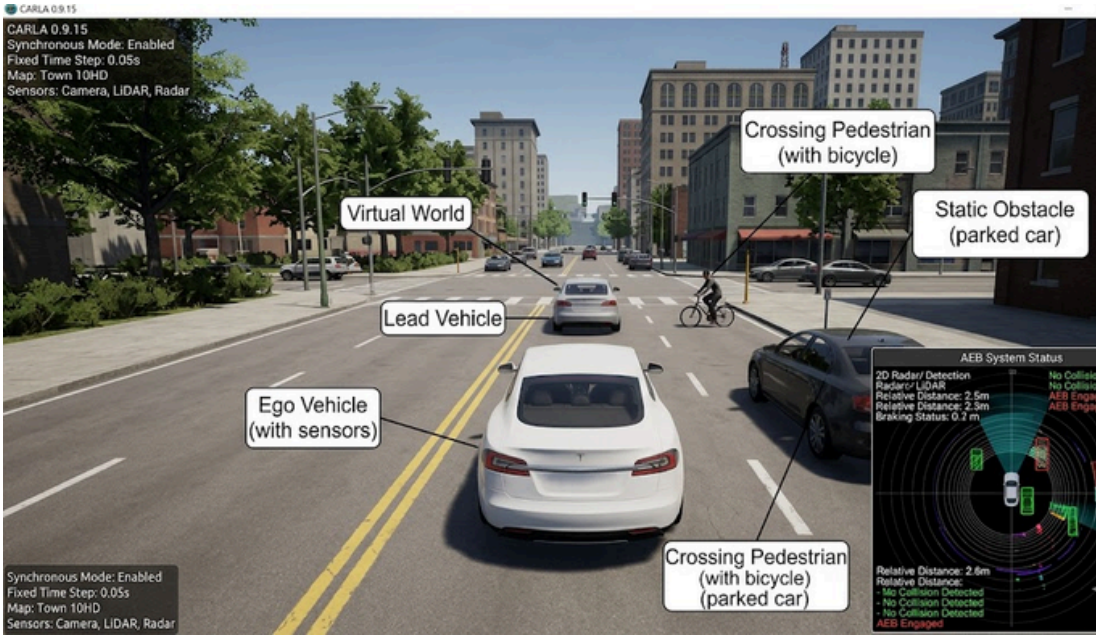


Figure 4.3. CARLA simulation environment

4.3.2 System Class Diagram

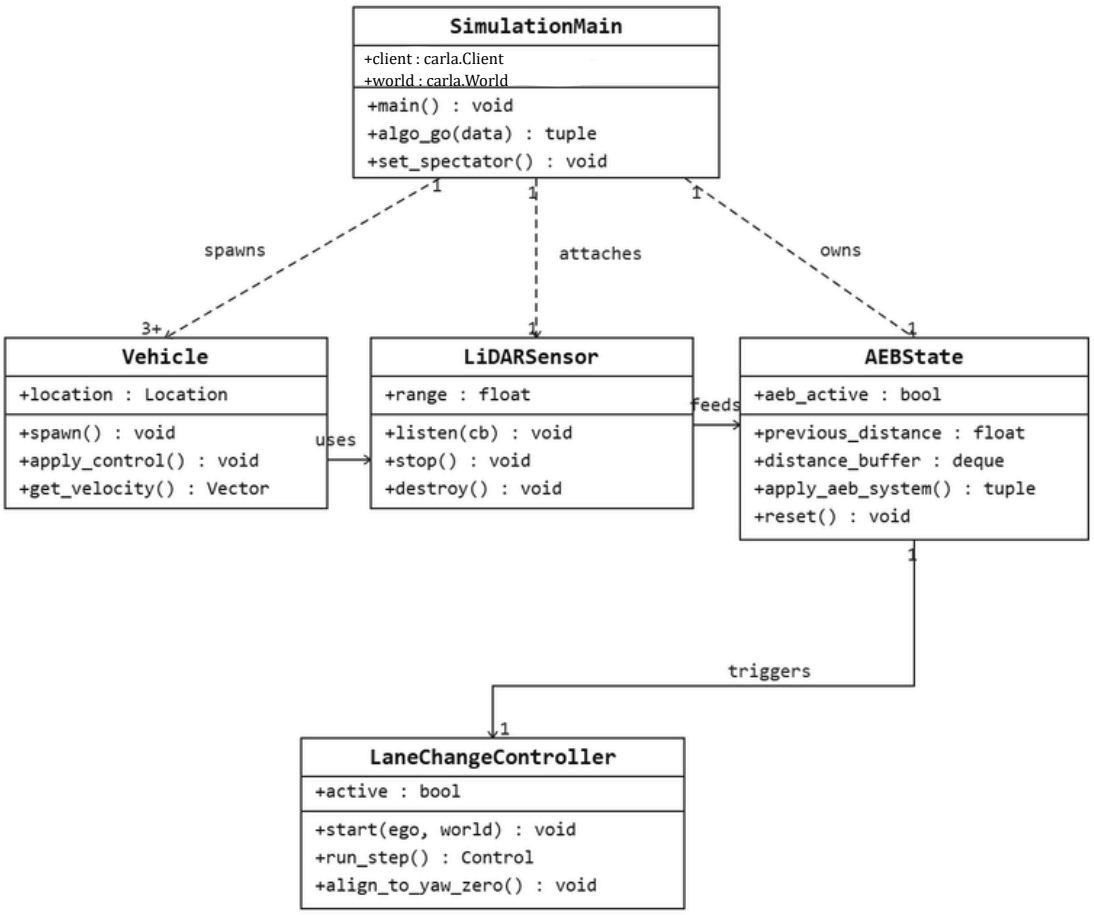


Figure 4.4. AEB CARLA Simulation Class Diagram

The class diagram in Figure 4.4 illustrates the main components of the AEB simulation implemented within the CARLA environment. The SimulationMain class acts as the central controller, responsible for initializing the simulation, spawning all actors, and running the main execution loop. It interacts with the Vehicle class, which represents all spawned actors in the scene including the ego vehicle and surrounding NPC vehicles. The LiDARSensor class is attached to the ego vehicle and continuously captures point cloud data to detect obstacles ahead. This data is passed to the AEBState class, which processes the sensor readings, estimates the relative speed, and computes the braking decision based on the Time-To-Collision (TTC). When a collision risk is detected and braking alone is insufficient, the LaneChangeController is triggered to perform an autonomous lane change manoeuvre, guiding the ego vehicle safely around the obstacle.

4.3.3 Scenario Description

This section documents the six simulation scenarios used to validate the AEB system in the CARLA autonomous driving simulator. The scenarios were selected by studying the Euro NCAP AEB Test Protocol (v4.3.1, February 2024) — the internationally recognised standard for AEB system testing used by automotive manufacturers and regulatory bodies worldwide — and choosing the subset most representative of the hazard conditions the system is designed to address

The Euro NCAP protocol defines two categories of AEB scenarios: Car-to-Car (C2C) scenarios involving vehicle-to-vehicle interactions, and Vulnerable Road User (VRU) scenarios involving pedestrians, cyclists, and motorcyclists. From the full catalogue of scenarios defined in the protocol, six were selected for CARLA implementation based on their relevance to the system's operating speed range (up to 60 km/h), sensor configuration (forward-facing LiDAR), and the AEB algorithm's FCW escalation logic

Each scenario is described in detail in the sections below. For every scenario, the following information is provided: the corresponding Euro NCAP scenario code and definition, the specific implementation in CARLA (actor configuration, spawn positions, speeds, and trigger conditions), the expected system behaviour from the AEB algorithm, and the actual observed result during testing.

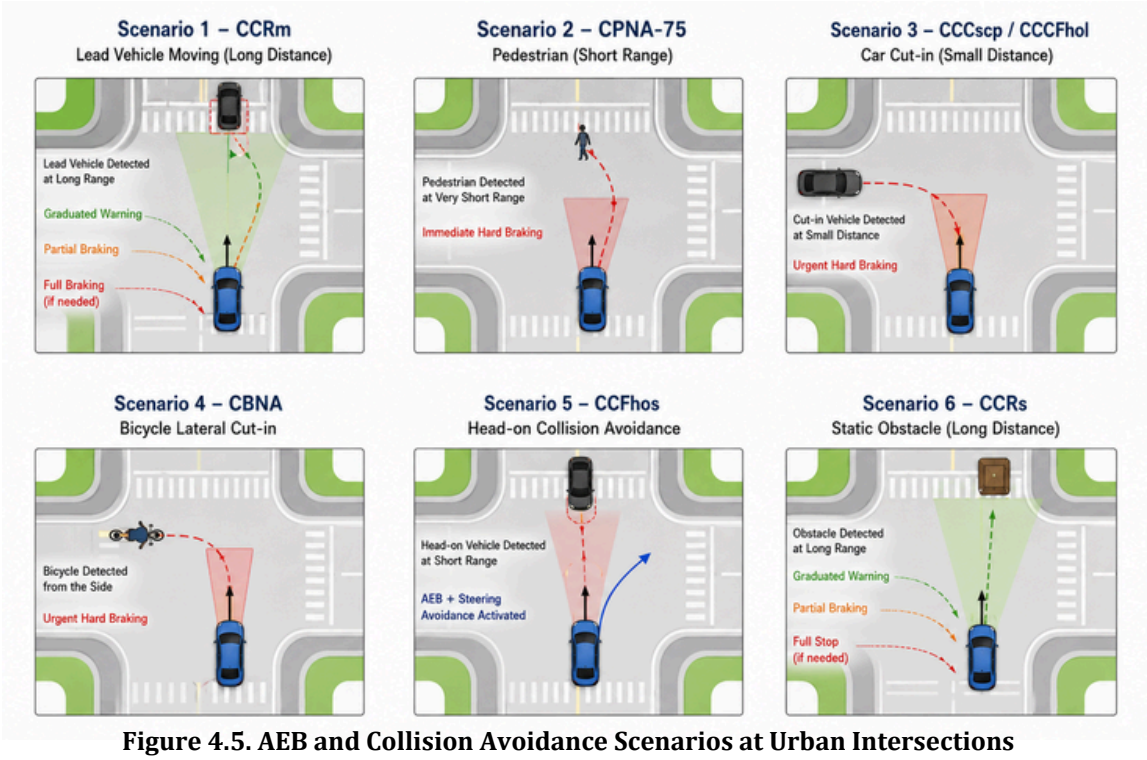


Figure 4.5. AEB and Collision Avoidance Scenarios at Urban Intersections

Figure 4.5 illustrates six simulated test scenarios that validate the AEB/AES collision avoidance logic. These scenarios test the system’s ability to respond to varied critical situations, including long-range vehicle approaches, lateral pedestrian and bicycle cut-ins, and head-on collisions, demonstrating graduated braking and steering avoidance functions.

4.4 AUTOSAR

4.4.1 AUTOSAR Introduction

AUTOSAR (AUTomotive Open System ARchitecture) is a global partnership of automotive manufacturers, suppliers, and technology companies established in 2003 with the objective of creating and standardizing an open software architecture for automotive electronic control units (ECUs). The standard was developed in response to the growing complexity of vehicle software, which had become increasingly difficult to develop, integrate, and reuse across different vehicle platforms and ECU hardware variants. By defining a common application interface and a layered software architecture, AUTOSAR enables software components to be developed independently of the underlying hardware and to be reused and exchanged between different vehicle manufacturers and suppliers without modification.

The AUTOSAR architecture exists in two variants that address different levels of automotive computing. The Classic Platform, which is the variant adopted in this project, targets deeply embedded, real-time systems with deterministic timing requirements — such as body control modules, powertrain controllers, and safety-critical ADAS functions including AEB. The Adaptive Platform, introduced later, targets high-performance computing environments running POSIX-based operating systems and is intended for applications such as automated driving and over-the-air software updates.

The Classic Platform organizes software into a strict layered model consisting of three main layers. The lowest layer is the Basic Software (BSW), which abstracts the microcontroller hardware and provides standardized services such as communication drivers, memory management, operating system scheduling, and diagnostic services. The BSW is further subdivided into the Microcontroller Abstraction Layer (MCAL), which contains hardware-specific drivers directly interfacing with the microcontroller peripherals; the ECU Abstraction Layer, which provides a hardware-independent interface above the MCAL; and the Services Layer, which offers operating system, communication, and memory services to the layers above. Above the BSW sits the Runtime Environment (RTE), which is the central middleware of the AUTOSAR architecture. The RTE is automatically generated by AUTOSAR toolchains from the system configuration and serves as the sole communication channel between application software components, decoupling them entirely from one another and from the BSW below. At the top of the stack, the Application Layer contains one or more Software Components (SWCs), each of which encapsulates a specific vehicle function and communicates with the rest of the system exclusively through well-defined ports — Sender-Receiver ports for data signals and Client-Server ports for function calls — accessed only through RTE-generated port access macros.

4.4.2 AUTOSAR Architecture

Software Components are formally described using the AUTOSAR XML schema (ARXML), which defines the component's ports, interfaces, internal behaviour, runnable entities, and timing events in a machine-readable format. This formal description enables AUTOSAR-compliant toolchains to automatically generate the RTE code, validate the system configuration, and map components to ECU hardware without manual intervention. Each SWC contains one or more runnable executable functions that are triggered by timing events, data reception events, or initialization events managed by the AUTOSAR Operating System (OS). The OS schedules these runnable according to the configured activation periods, ensuring deterministic real-time execution with defined worst-case execution times.

In the context of this project, the AUTOSAR Classic Platform provides the software architecture framework within which the AEB algorithm is implemented, structured, and validated. The algorithm is encapsulated as a single Application SWC Algorithm SWC whose ports, runnable, and inter-runnable variables are formally described in the project's ARXML file. The SWC is deployed on a virtual ECU based on the Infineon AURIX TriCore microcontroller architecture, with the AUTOSAR OS managing the cyclic scheduling of the four runnable at a fixed period of 50 milliseconds. This architecture ensures that the AEB system meets the real-time, portability, and functional safety requirements expected of a production-grade automotive safety function.

•

4.4.3 Software Component Architecture: Three-SWC Design

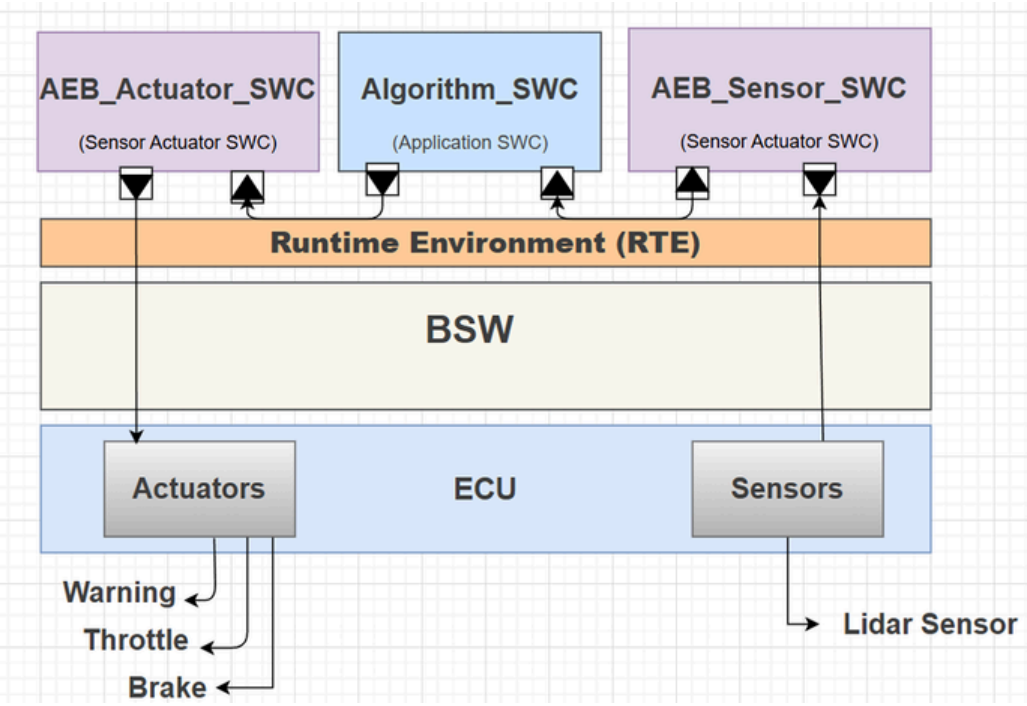
The single-SWC description given in Section 4.4.2 reflects an early design iteration of the project. As the integration work progressed, the team identified that concentrating sensor acquisition, algorithmic processing, and actuation buffering inside one monolithic *Algorithm_SWC* violated the AUTOSAR separation-of-concerns principle and made the component difficult to test in isolation. Consequently, the final architecture decomposes the AEB function into **three independent Software Components**, each with a single, well-defined responsibility, connected exclusively through RTE ports. This section documents the final, as-built architecture that was implemented, simulated, and used to generate all results presented later in this report.
The three Software Components are:

- **AEB_Sensor_SWC** — a Sensor/Actuator-type SWC responsible exclusively for receiving raw ego-vehicle speed and raw LiDAR-derived distance values arriving from the SIL Kit bus, and republishing them on its Provide-Ports for consumption by the algorithm component.

- **Algorithm_SWC** — an Application-type SWC that hosts the AEB decision logic itself: distance smoothing, relative-speed estimation, Time-to-Collision computation, Forward Collision Warning classification, and the brake/throttle decision block.

- **AEB_Actuator_SWC** — a Sensor/Actuator-type SWC that acts as a change-detection output buffer. It withholds redundant actuation commands and forwards a signal to SIL Kit only when its value differs from the last value sent, reducing unnecessary CAN bus traffic.

This three-component decomposition follows the classic *Sense → Think → Act* pattern that is standard practice in automotive software design, and it maps cleanly onto the perception, decision, and actuation layers introduced conceptually in Section 4.1. Each SWC is described structurally in the subsections that follow, and each is backed by a corresponding ARXML definition containing its ports, interfaces, internal behaviour, and runnable entities.

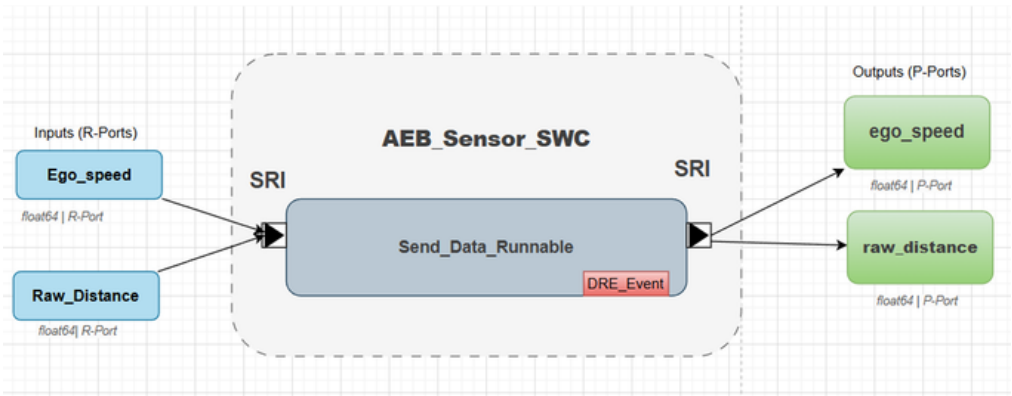


4.4.3.1 AEB_Sensor_SWC

The AEB_Sensor_SWC is modelled as a SENSOR-ACTUATOR-SW-COMPONENT-TYPE in the ARXML description. It exposes two Require-Ports, RP_Ego_speed and RP_Raw_Distance, which receive data published on the SIL Kit bus by the CARLA-side gateway. It exposes two corresponding Provide-Ports, PP_ego_speed and PP_raw_distance, through which the same values are forwarded into the RTE for the algorithm component to consume.

The internal behaviour of this SWC is intentionally minimal. A single runnable, Send_Data_Runnable, is configured against a DATA-RECEIVED-EVENT bound to RP_Ego_speed. Whenever a new ego-speed sample arrives on the bus, the RTE Operating System triggers this runnable, which reads both the ego speed and the raw distance from its Require-Ports and immediately writes them, unchanged, to its Provide-Ports. A second runnable, Runnable_Init_Sensor, is triggered once at start-up by an INIT-EVENT and is retained as an extension point for future sensor diagnostics, even though no initialisation logic is currently required for a pure pass-through component.

Designing the sensor interface as a separate SWC, rather than reading SIL Kit signals directly inside the algorithm component, isolates the AEB logic from the communication middleware. If the underlying transport were changed — for example, from SIL Kit CAN messages to a different bus technology — only AEB_Sensor_SWC would need to be modified; Algorithm_SWC would remain entirely unaffected, since it only ever interacts with standardised AUTOSAR ports.

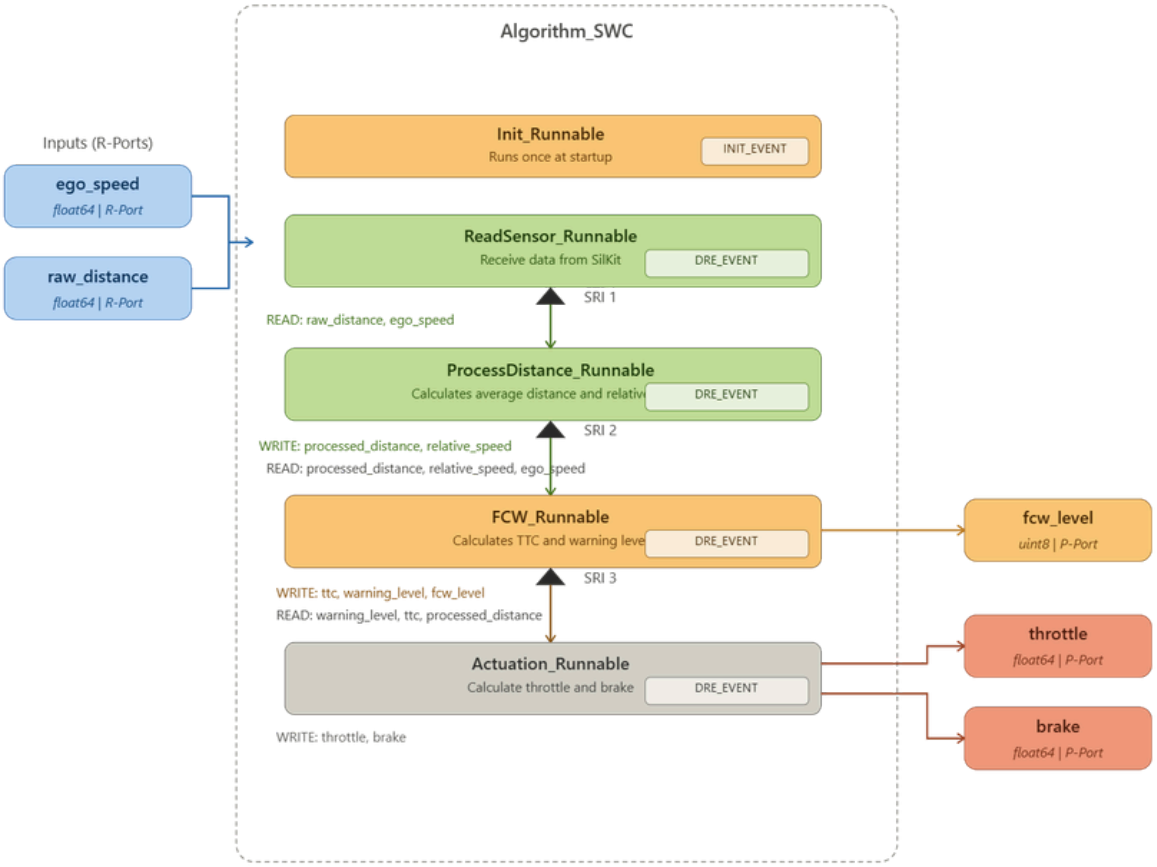


4.4.3.2 Algorithm_SWC

The Algorithm_SWC is the computational core of the system and is modelled as an APPLICATION-SW-COMPONENT-TYPE. It receives ego_speed and raw_distance from AEB_Sensor_SWC through two Require-Ports, and exposes throttle, brake, and fcw_level on three Provide-Ports that connect to AEB_Actuator_SWC. In addition to these externally visible ports, the component defines a set of internally looped Sender-Receiver ports — processed_distance, relative_speed, ttc, and warning_level — that are not exposed outside the SWC but instead carry intermediate results between its own runnables. This use of inter-runnable variables, rather than ordinary local variables shared across functions, keeps every data exchange inside the component visible to the RTE and therefore traceable and schedulable.

The component's internal behaviour, Behavior_AEB, defines four runnables. Runnable_Init executes once at start-up, initialising the persistent AEBState structure — the distance circular buffer, the previous-distance memory, the low-pass filter state, and the AEB-active flag — to a known zero state. The remaining three runnables are each bound to a TIMING-EVENT configured with a fixed period of 50 milliseconds, giving the AEB function a deterministic 20 Hz decision rate that matches the CARLA simulation's fixed time step described in Section 4.3.

- **Runnable_ProcessDistance** reads *raw_distance* and *ego_speed* from the sensor component, applies the circular-buffer smoothing filter, derives the relative closing speed via low-pass filtering, and writes *processed_distance* and *relative_speed* for the next runnable to consume.
- **Runnable_FCW** reads the smoothed distance and relative speed together with *ego_speed*, computes the Time-to-Collision, classifies it into one of four Forward Collision Warning levels, and writes *ttc*, *warning_level*, and the externally visible *fcw_level*.
- **Runnable_Actuation** reads the warning level, the TTC value, and the processed distance, applies the brake/throttle decision block, and writes the final *throttle* and *brake* values to the Provide-Ports connected to *AEB_Actuator_SWC*.



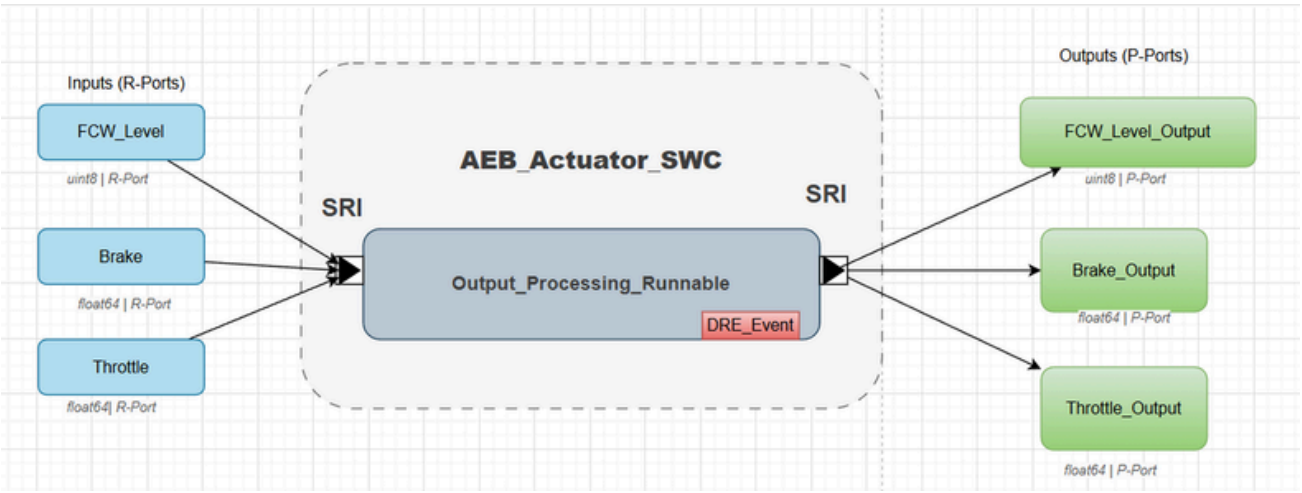
4.4.3.3 AEB_Actuator_SWC

The AEB_Actuator_SWC closes the loop back toward the simulation. It is modelled as a second SENSOR-ACTUATOR-SW-COMPONENT-TYPE and receives throttle, brake, and fcw_level from Algorithm_SWC on three Require-Ports. It exposes three corresponding Provide-Ports — throttle_output, brake_output, and fcw_level_output — which are the signals actually published toward SIL Kit and, from there, applied to the CARLA ego vehicle.

What distinguishes this component from a simple pass-through is its use of Per-Instance Memory (PIM) to implement change-detection buffering. Three persistent memory cells — LastSent_Throttle, LastSent_Brake, and LastSent_FCWLevel — record the most recent value that was actually transmitted toward CARLA for each signal. At start-up, Runnable_Init_Actuator loads these cells with sentinel values that lie outside the physically valid range of each signal: −1.0 for throttle and brake (whose valid range is [0, 1]), and 255 for the FCW level (whose valid range is [0, 3]). This guarantees that the very first genuine sample received from Algorithm_SWC is always recognised as a change and is always forwarded.

The single data-processing runnable, Output_Processing_Runnable, is triggered by a DATA-RECEIVED-EVENT bound to the fcw_level Require-Port — the last value written by Runnable_FCW each cycle — so that the runnable fires only after all three upstream values are guaranteed to be fresh. On each invocation it reads the three incoming values, compares each one individually against its corresponding PIM cell, and writes to the matching Provide-Port — and updates the PIM — only for the signals whose value has actually changed since the previous cycle. Signals that are unchanged are simply skipped, and no redundant message is sent over the SIL Kit bus.

This design has two practical benefits within the safety-critical context of an AEB system. First, it reduces bus load by suppressing repeated transmission of an unchanging brake command, which matters when the same CAN channel is shared with other monitored signals and visualised live on the CANoe dashboard described in Section 4.2. Second, it provides a deterministic, easily testable definition of what “sending an actuation command” means: a command is sent if, and only if, the computed value differs from what was previously sent, which simplifies later verification of actuator behaviour during the test scenarios described in Section 4.3.3.



4.4.3.4 Port and Interface Summary

Table 4.1 summarises the complete set of Sender-Receiver interfaces, their owning Software Component, port direction, and underlying AUTOSAR implementation data type, exactly as declared in the project's ARXML description. This table provides a single reference point that ties together the three SWC descriptions given above.

Interface	Data Element	Type	Owning SWC (P-Port)	Consuming SWC (R-Port)
SRI_EgoSpeed	ego_speed	float64	AEB_Sensor_SWC	Algorithm_SWC
SRI_RawDistance	raw_distance	float64	AEB_Sensor_SWC	Algorithm_SWC
SRI_ProcessedDistance	processed_distance	float64	Algorithm_SWC (internal)	Algorithm_SWC (internal)
SRI_RelativeSpeed	relative_speed	float64	Algorithm_SWC (internal)	Algorithm_SWC (internal)
SRI_TTC	ttc	float64	Algorithm_SWC (internal)	Algorithm_SWC (internal)
SRI_WarningLevel	warning_level	uint8	Algorithm_SWC (internal)	Algorithm_SWC (internal)
SRI_Throttle	throttle	float64	Algorithm_SWC	AEB_Actuator_SWC
SRI_Brake	brake	float64	Algorithm_SWC	AEB_Actuator_SWC
SRI_FCWLevel	fcw_level	uint8	Algorithm_SWC	AEB_Actuator_SWC
SRI_ThrottleOutput	throttle_output	float64	AEB_Actuator_SWC	SIL Kit (external)
SRI_BrakeOutput	brake_output	float64	AEB_Actuator_SWC	SIL Kit (external)
SRI_FCWLevelOutput	fcw_level_output	uint8	AEB_Actuator_SWC	SIL Kit (external)

4.5 User Interface and Monitoring Dashboard

Although the AEB function itself runs without any human interface — it is a fully automated safety function — the project incorporates a diagnostic layer that allows the development team to observe the system's internal state in real time during simulation. This monitoring capability is essential for a safety-critical system: without visibility into intermediate values such as TTC and FCW level, validating the correctness of the braking decision would be limited to observing only the vehicle's final motion in CARLA, which is insufficient for root-cause analysis when a scenario does not behave as expected.

4.5.1 CANoe Dashboard

As introduced in Section 4.2, the CAN bus carrying signals between *Algorithm_SWC* and the SIL Kit gateway is also tapped by a Vector CANoe panel configured specifically for this project. The CANoe dashboard subscribes passively to the same CAN messages exchanged between the AUTOSAR stack and SIL Kit and renders them as live numeric and graphical widgets, without interfering with the timing of the simulation itself.

The dashboard is organised around the signals most relevant to verifying AEB behaviour:

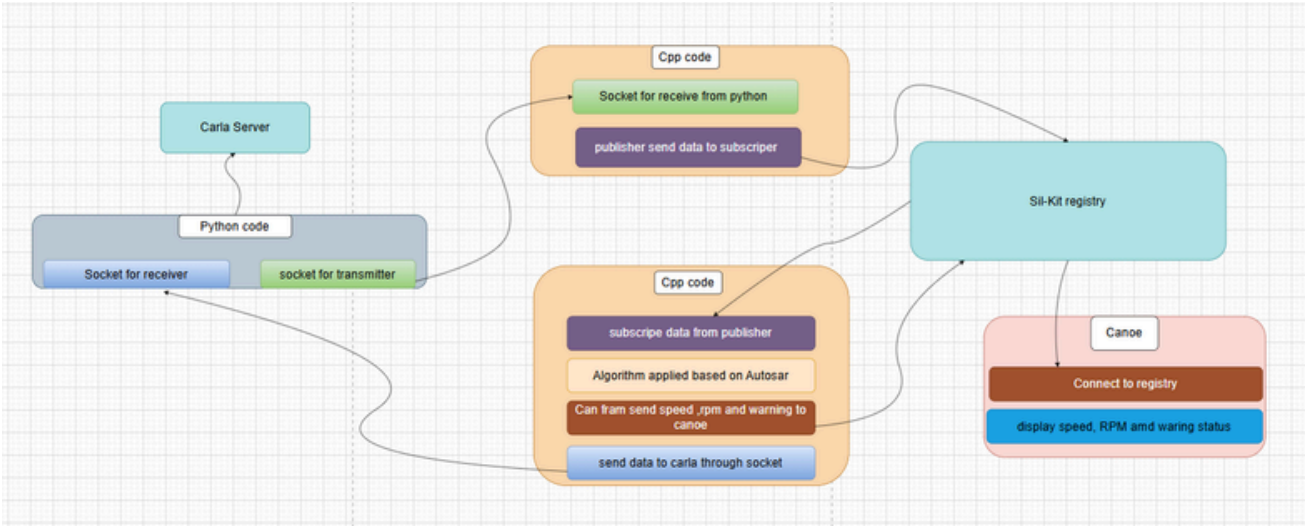
- **Vehicle speed and engine RPM** — displayed as analogue-style gauges, giving an immediate visual sense of whether the ego vehicle is decelerating in response to a detected hazard.
- **Raw and processed distance** — plotted as a time-series trace, allowing the smoothing effect of the circular buffer described in Section 4.4.3.2 to be visually confirmed against the raw LiDAR-derived input.
- **FCW level** — shown as a four-state indicator (0–3), colour-coded from green (no warning) through yellow and orange to red (critical braking), mirroring the escalation logic of Figure 4.11.
- **Throttle and brake output** — displayed as percentage bars reflecting the values actually transmitted by AEB_Actuator_SWC after change-detection filtering, rather than the raw values computed inside Algorithm_SWC, so that the dashboard reflects exactly what reaches the vehicle.

4.7 Full System Architecture and Data Flow

Figure illustrates the overall architecture of the proposed Software-in-the-Loop (SiL) Autonomous Emergency Braking (AEB) system and the interaction between its main components. The system begins with the CARLA simulator, which generates real-time vehicle and environmental data. A Python-based gateway acts as an interface between CARLA and the AUTOSAR environment. The gateway receives simulation data through a dedicated socket and transmits the relevant sensor information, such as vehicle speed and obstacle distance, to a C++ SIL Kit publisher participant.

The publisher participant receives the data through a socket connection and publishes it to the SIL Kit communication bus via the SIL Kit Registry. On the receiving side, an AUTOSAR-based C++ participant subscribes to the published messages and processes them using the implemented AEB algorithm. The algorithm analyzes the incoming sensor data to determine the appropriate warning or braking action based on the current driving situation. The resulting outputs, including vehicle speed, engine RPM, and warning status, are then transmitted over a CAN frame and made available to other participants connected to the SIL Kit network.

A CANoe node is connected to the SIL Kit Registry to emulate the vehicle communication network and monitor system behavior. CANoe receives and displays the transmitted signals, providing real-time visualization of speed, RPM, and warning information for validation and debugging purposes. Additionally, the AUTOSAR participant sends processed control information back to the Python gateway through a socket connection, enabling closed-loop interaction with the CARLA simulation. This architecture demonstrates seamless integration between CARLA, SIL Kit, AUTOSAR software components, and CANoe, providing a realistic environment for developing, testing, and validating advanced driver assistance functions before deployment on physical hardware.



4.8 Technology Stack

Table 4.3 consolidates every tool, framework, and language used in the design and implementation of the system described throughout this chapter, organised by the architectural layer each one supports.

Layer	Technology	Role in the System
Environment Simulation	CARLA Simulator	Generates the virtual driving environment, ego-
Gateway / Bridging	C++ with TCP Sockets	Bridges CARLA's Python-based client API to the SIL
Communication Middleware	Vector SIL Kit	Provides the CAN-based publish/subscribe bus
Software Architecture Standard	AUTOSAR Classic Platform	Defines the layered structure, port-based
AUTOSAR Modelling / RTE Generation	Artop (Eclipse-based AUTOSAR Tool Platform)	Used to author the ARXML component descriptions
Application Logic Language	C (C99)	Implementation language for the algorithmic core
Build Toolchain	GCC (MinGW on Windows)	Compiles the SWC implementations together
Development Environment	Eclipse CDT	Primary IDE used for C development, debugging,
Diagnostics / Monitoring	Vector CANoe	Hosts the live dashboard panel used to observe CAN
Manual Override Input	Joystick (via CARLA input API)	Allows manual driving input to be injected into the
Version Control	Git / GitHub	Manages source history and coordinates parallel work

4.9 Synchronization, Timing, and Data Consistency

Section 4.1 introduced the importance of timing determinism for a safety-critical AEB function. This section explains concretely how deterministic behavior is achieved across the boundary between the CARLA simulation, the SIL Kit middleware, and the three AUTOSAR Software Components.

4.9.1 Fixed-Step Synchronous Simulation

As described in Section 4.3, CARLA is configured to run in synchronous mode with a fixed time step of 0.05 seconds. In this mode, CARLA does not advance its internal physics or sensor sampling until it receives an explicit tick acknowledgement from the connected client, rather than advancing in real wall-clock time. This guarantees that exactly one LiDAR sample and one vehicle-speed sample are produced per simulated 50 ms interval, with no risk of the simulator silently dropping or duplicating a frame under heavy computational load on the host machine.

4.9.2 Event-Driven Data Processing

The fixed-step simulation ensures that sensor measurements are generated at predictable intervals, while the AUTOSAR application layer processes these measurements using Data Received Events (DREs). Whenever a new sensor value arrives through the RTE, the corresponding runnable in AEB_Sensor_SWC is activated immediately. The received data is then forwarded through the AUTOSAR communication path, triggering the processing chain in Algorithm_SWC and ultimately producing an actuation decision. This event-driven approach eliminates the need for periodic polling and ensures that each computation is based on the most recently available sensor information.

4.9.3 Message Latency and the SIL Kit Bus

The SIL Kit bus introduces a small, bounded transport latency between the moment a value is published by one participant and the moment it becomes visible to another. In the data-received-event-driven design of AEB_Sensor_SWC (Section 4.4.3.1) and AEB_Actuator_SWC (Section 4.4.3.3), this latency is handled implicitly: rather than polling for new data at a fixed instant, each component's runnable is activated directly by the arrival event itself, so the runnable always operates on the freshest value available at the moment it is scheduled, regardless of small jitter in exact arrival timing.

4.9.4 Toward Hardware-in-the-Loop

The architecture described above is readily extensible toward Hardware-in-the-Loop (HIL) testing, as noted in Section 4.1. Because the three SWCs interact with the rest of the system exclusively through RTE ports and SIL Kit messages — never through direct function calls into CARLA or the gateway — replacing the simulated SIL Kit bus with a physical CAN transceiver connected to a real microcontroller running the same Basic Software stack would require no modification to the SWC implementations described in Section 4.4.3. The event-driven communication model and RTE port interfaces remain unchanged, allowing the same application software to operate in both simulation and real hardware environments with minimal integration effort.

4.10 Chapter Summary

This chapter has presented the complete design of the AEB integration project, progressing from the high-level architecture connecting CARLA, SIL Kit, and the AUTOSAR stack, through the CARLA simulation environment and its six validation scenarios, to the detailed internal structure of the three Software Components that together implement the AEB function. The *AEB_Sensor_SWC*, *Algorithm_SWC*, and *AEB_Actuator_SWC* were each described in terms of their ports, internal behaviour, and runnables, and the RTE mapping connecting them was documented alongside the sequence and activity diagrams that capture the system's dynamic behaviour. The chapter concluded with the diagnostic dashboards used to monitor the system and the complete technology stack, and explained how the fixed-step synchronisation between CARLA and the AUTOSAR task schedule provides the deterministic timing required by a safety-critical function.

Chapter (5)
Methodology

5.1 Introduction

The successful development of an Autonomous Emergency Braking (AEB) system requires more than the implementation of an effective braking algorithm. It also requires a structured development process capable of managing multiple technologies, coordinating software integration activities, and ensuring reliable validation of system behavior. Since the proposed solution combines a high-fidelity simulation environment, AUTOSAR-based software architecture, communication middleware, and automotive diagnostic tools, a systematic methodology was essential to guide the project from concept to implementation.

This chapter presents the methodology adopted throughout the development of the project. It describes the software development model used by the team, the planning and execution phases, the tools and technologies employed during implementation, and the distribution of tasks among team members. The chapter also explains how integration, testing, and validation activities were organized to ensure that the final system satisfied both functional and non-functional requirements.
The methodology was designed to support incremental development and continuous verification. Instead of attempting to implement the complete system in a single stage, the project was divided into multiple development iterations. Each iteration focused on a specific subsystem, allowing components to be tested individually before being integrated into the complete Software-in-the-Loop (SiL) environment.

The project development process began with a detailed literature review covering Autonomous Emergency Braking systems, Advanced Driver Assistance Systems (ADAS), AUTOSAR architecture, and automotive simulation platforms. This research phase established the theoretical foundation required to understand modern automotive software development practices and identify suitable technologies for implementation.

Following the research phase, the team proceeded to the design and implementation stages. The CARLA simulator was configured to create realistic driving scenarios, while AUTOSAR Software Components were developed to implement the AEB decision logic. Vector SIL Kit was then integrated to enable communication between the simulation environment and the virtual ECU implementation. Finally, CANoe was used to monitor and validate the transmitted signals during execution.
The adopted methodology emphasized modularity, reusability, and verification. By separating the project into distinct layers and components, each subsystem could be developed, tested, and improved independently. This approach reduced integration complexity and improved overall system reliability.

The remainder of this chapter details the development methodology, project planning activities, software tools, implementation workflow, team organization, and quality assurance practices followed throughout the project lifecycle.

5.2 Development Methodology

5.2.1 Selection of Development Model

Selecting an appropriate development methodology was an important factor in the success of the project. The project involved multiple interconnected technologies, including CARLA simulation, AUTOSAR software development, TCP socket communication, SIL Kit integration, CAN communication, and diagnostic visualization through CANoe. Because of this complexity, a development model capable of supporting incremental progress and continuous integration was required.

Several software development methodologies were evaluated before implementation began. Traditional models such as the Waterfall model provide a sequential workflow where each phase must be completed before the next phase begins. While this approach is suitable for projects with stable requirements, it becomes less effective when continuous experimentation and integration are required.

Automotive simulation projects often involve unforeseen challenges related to software compatibility, communication protocols, and performance optimization. As a result, the development team determined that a rigid sequential methodology would introduce unnecessary risk and limit flexibility.

Consequently, an Agile-inspired development approach was adopted. Agile methodologies emphasize iterative development, continuous testing, and frequent evaluation of progress. Rather than treating the project as a single large task, Agile divides development into smaller manageable iterations, allowing improvements to be incorporated throughout the project lifecycle.

The Agile approach provided several advantages for this project:

- Early identification of integration issues.
- Continuous validation of software functionality.
- Improved flexibility when adapting system architecture.
- Incremental delivery of project features.
- Better collaboration between team members.
- Reduced risk of late-stage integration failures.

These advantages were particularly important because the project combined both automotive software engineering concepts and simulation technologies that required extensive experimentation and validation.

5.2.2 Agile Development Process

The adopted Agile process divided the project into several development iterations. Each iteration focused on a specific subsystem and concluded with testing and evaluation activities. This incremental strategy ensured that each module was functioning correctly before progressing to the next development stage.

- The first iteration focused primarily on research and technology familiarization. During this phase, the team studied AUTOSAR Classic Platform concepts, Software Components, Runtime Environment generation, and automotive communication protocols. Simultaneously, team members explored the capabilities of the CARLA simulator and Vector SIL Kit.

- The second iteration concentrated on building the simulation environment. Vehicle models, sensors, road networks, and test scenarios were configured within CARLA. Particular attention was given to LiDAR sensor configuration because obstacle detection forms the foundation of the AEB system.

- The third iteration focused on AUTOSAR software development. During this phase, ARXML descriptions were created, Software Components were modeled, and runnable entities were implemented. The project architecture evolved from an initial monolithic component design into a three-component architecture consisting of AEB_Sensor_SWC, Algorithm_SWC, and AEB_Actuator_SWC.

- The fourth iteration addressed communication and integration challenges. SIL Kit participants were developed and configured to exchange messages through the communication bus. TCP socket communication was established between CARLA and the middleware layer, enabling real-time data exchange.

- The fifth iteration focused on validation and performance assessment. Test scenarios were executed repeatedly to verify that the AEB algorithm responded correctly to different driving situations. CANoe dashboards were used to observe system behavior and verify signal transmission throughout the communication chain.

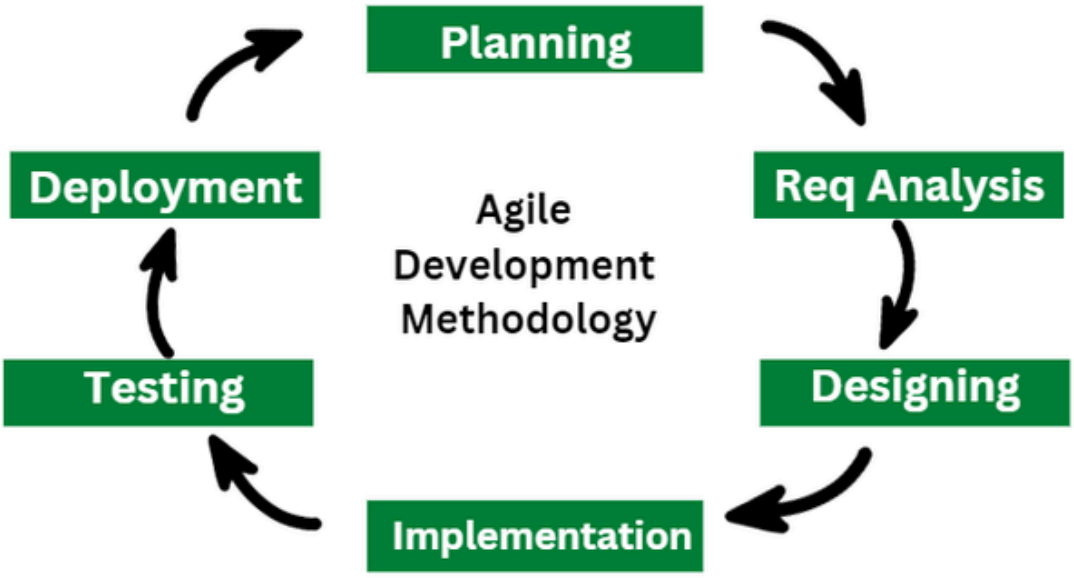
At the conclusion of each iteration, the results were reviewed and any identified issues were documented. Necessary modifications were then incorporated into the next iteration, ensuring continuous improvement of the overall system.

5.2.3 Incremental System Integration

A key principle of the adopted methodology was incremental integration. Rather than integrating all system components simultaneously, integration was performed progressively. The first integration step involved connecting CARLA with the Python gateway. Once stable communication was achieved, SIL Kit participants were added to the architecture. AUTOSAR Software Components were then integrated into the communication chain. Finally, CANoe was connected for monitoring and validation.

This staged integration strategy significantly reduced debugging complexity. Problems could be isolated to specific subsystems rather than requiring investigation across the entire architecture.

Furthermore, incremental integration allowed performance bottlenecks to be identified early in development. Communication latency, message synchronization, and data consistency could be evaluated independently before full system deployment.
The methodology therefore provided a structured and manageable path from initial concept development to complete system integration while maintaining reliability and traceability throughout the project lifecycle.



5.3 Project Planning

5.3.1 Project Scope and Objectives

Before the implementation phase began, a clear project scope was defined to establish development boundaries and ensure that all team members shared a common understanding of the project's objectives. The primary goal of the project was to develop and validate an Autonomous Emergency Braking (AEB) system within a Software-in-the-Loop (SiL) environment using AUTOSAR-based software architecture.

The project sought to combine multiple technologies commonly used in modern automotive development environments, including the CARLA simulator, AUTOSAR Classic Platform concepts, Vector SIL Kit communication middleware, and CANoe diagnostic tools. By integrating these technologies, the project aimed to create a realistic virtual platform capable of evaluating AEB functionality before deployment on physical hardware.

The specific objectives of the project were:

- Develop an AUTOSAR-compliant implementation of an AEB algorithm.
- Create realistic driving scenarios using the CARLA simulator.
- Simulate LiDAR-based obstacle detection.
- Implement vehicle risk assessment using Time-to-Collision (TTC) calculations
- Generate multi-level Forward Collision Warnings (FCW).
- Enable automatic braking decisions based on collision risk.
- Establish communication between simulation and AUTOSAR software using SIL Kit.
- Monitor system behavior through CANoe dashboards.
- Validate the complete system through multiple test scenarios.

By clearly defining these objectives, the project team was able to focus development efforts on measurable deliverables while avoiding unnecessary scope expansion.

5.3.2 Work Breakdown Structure (WBS)

To facilitate effective project management, the entire development process was decomposed into smaller and more manageable tasks using a Work Breakdown Structure (WBS). The WBS divided the project into logical activities that could be assigned, monitored, and completed independently.

The top-level project structure consisted of six primary work packages:

1. Research and Literature Review
2. Simulation Environment Development
3. AUTOSAR Software Development
4. Communication Integration
5. Testing and Validation
6. Documentation and Reporting

The Research and Literature Review package included studying AEB systems, AUTOSAR architecture, CAN communication, SIL Kit middleware, and automotive safety concepts.

The Simulation Environment Development package focused on configuring the CARLA simulator, creating test scenarios, implementing sensors, and generating environmental data.

The AUTOSAR Software Development package included ARXML modeling, Software Component creation, runnable implementation, and Runtime Environment integration.

The Communication Integration package covered TCP socket communication, SIL Kit participant development, CAN message configuration, and communication validation.

The Testing and Validation package involved executing simulation scenarios, analyzing system performance, measuring communication behavior, and validating algorithm correctness.

Finally, the Documentation package included preparation of reports, diagrams, experimental results, and project presentation materials.

The use of a Work Breakdown Structure improved project organization and enabled more accurate tracking of progress throughout development.

5.3.3 Development Timeline

The project was executed over multiple academic semesters. To ensure efficient utilization of available time, the development activities were organized according to a predefined timeline. The timeline consisted of six major phases:

Phase	Duration	Activities	Milestone
Phase 1: Research and Technology Exploration	3 Weeks	• Literature review • Technology evaluation • Requirements analysis • Learning AUTOSAR concepts	**Literature Review Completion**
Phase 2: System Design	4 Weeks	• Architecture definition • Component modeling • Interface design • Communication planning	**System Design Completion**
Phase 3: Simulation Development	5 Weeks	• CARLA setup • Scenario creation • Sensor configuration • Data acquisition development	**Simulation Environment Operational**
Phase 4: AUTOSAR Development	5 Weeks	• ARXML creation • SWC implementation • Runnable development • RTE integration	**AUTOSAR SWCs Implemented**
Phase 5: Integration and Testing	4 Weeks	• SIL Kit integration • Socket communication testing • CANoe monitoring setup • Validation scenario execution	**System Integration Completed**
Phase 6: Documentation and Finalization	3 Weeks	• Results analysis • Report writing • Presentation preparation • Final system demonstration	**Validation and Documentation Completed**

This figure illustrates the planned execution schedule of the project, showing the duration and sequence of each development phase from initial research activities through final documentation and validation. The timeline demonstrates the incremental development strategy adopted throughout the project lifecycle and highlights the major milestones achieved at each stage.

Table 5.2: Weekly Project Schedule (Gantt Representation)

Project Phase	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15	W16	W17	W18	W19	W20	W21	W22	W23	W24
Research & Technology Exploration	■	■	■																					
System Design				■	■	■	■																	
Simulation Development								■	■	■	■	■												
AUTOSAR Development													■	■	■	■	■	■						
Integration & Testing																			■	■	■			
Documentation & Finalization																						■	■	■

5.3.4 Project Milestones

Project milestones were established to evaluate progress and ensure that critical deliverables were completed according to schedule.

Milestone 1: Literature Review Completion
The successful completion of research activities and technology selection marked the first major milestone. At this stage, project requirements were fully understood and implementation technologies were finalized.

Milestone 2: CARLA Environment Operational
The second milestone was achieved when the simulation environment was successfully configured and capable of generating realistic sensor data.

Milestone 3: AUTOSAR Software Components Implemented
This milestone represented the successful implementation of the three Software Components:

- AEB_Sensor_SWC
- Algorithm_SWC
- AEB_Actuator_SWC

Milestone 4: SIL Kit Communication Established
This milestone was achieved when data could be successfully exchanged between the simulation environment and AUTOSAR software components through the SIL Kit communication framework.

Milestone 5: Complete System Integration
The fifth milestone marked the successful integration of CARLA, SIL Kit, AUTOSAR, and CANoe into a unified Software-in-the-Loop environment.

Milestone 6: Validation Completion
The final milestone was reached when all simulation scenarios were executed successfully and the AEB system demonstrated correct behavior under all tested conditions.

5.3.5 Risk Assessment and Mitigation

Complex engineering projects inevitably involve technical risks that may affect project progress and system performance. Therefore, a risk assessment process was conducted during project planning to identify potential challenges and establish mitigation strategies.

One significant risk involved communication failures between system components. Since the architecture relied on multiple communication layers, including TCP sockets, SIL Kit, and CAN messaging, any communication disruption could prevent proper system operation. To mitigate this risk, communication modules were tested independently before integration.

Another risk involved software compatibility issues. The project combined tools originating from different vendors and development ecosystems. Version mismatches or configuration inconsistencies could lead to unexpected integration problems. This risk was addressed through careful tool selection and extensive compatibility testing.

Performance limitations represented an additional concern. Real-time automotive applications require predictable timing behavior. Excessive communication delays or processing overhead could reduce system responsiveness. To mitigate this risk, communication latency and processing performance were continuously monitored throughout development.

Algorithmic risks were also considered. Incorrect TTC calculations or warning thresholds could produce inappropriate braking decisions. To address this challenge, multiple test scenarios were executed repeatedly and algorithm outputs were compared against expected results.

Finally, project schedule risks were managed through incremental development and continuous progress reviews. Dividing the project into smaller milestones allowed delays to be detected early and corrective actions to be implemented before they affected the final delivery schedule.

The combination of structured planning, milestone tracking, and proactive risk management contributed significantly to the successful completion of the project.

5.4 Implementation Workflow

5.4.1 System Development Workflow

Following the planning and design activities described in previous sections, the implementation phase was carried out through a structured workflow that guided the development of all project components. The workflow was designed to ensure smooth integration between the simulation environment, communication middleware, AUTOSAR software architecture, and monitoring tools.

The implementation process began with the configuration of the CARLA simulation environment. Vehicle models, road networks, sensors, and traffic actors were created and validated before proceeding to software integration activities. Once the simulation environment was operational, the required sensor outputs were identified and prepared for transmission to the AUTOSAR environment.

The next stage involved the development of the communication gateway responsible for transferring data between CARLA and the SIL Kit communication infrastructure. TCP socket communication was selected because it provided a simple and efficient mechanism for exchanging data between Python-based simulation modules and C++ AUTOSAR participants.

After establishing communication channels, AUTOSAR Software Components were implemented according to the architecture presented in Chapter 4. Each component was developed independently and tested using predefined input signals before being integrated into the complete system.

Following software implementation, CAN communication parameters were configured and validated using SIL Kit and CANoe. The transmitted signals were monitored continuously to verify correct message formatting, timing behavior, and data consistency.

The final implementation stage involved full system integration, where all components operated simultaneously within the Software-in-the-Loop environment. Multiple simulation scenarios were executed to verify the interaction between sensing, decision-making, and actuation modules.

5.4.2 Software Component Implementation

The implementation of the AUTOSAR software architecture followed the three-component design adopted during system development. Each Software Component was developed independently and assigned a specific functional responsibility.

The AEB_Sensor_SWC was implemented as the interface between external sensor information and the AUTOSAR Runtime Environment. The component receives ego-vehicle speed and obstacle distance values from SIL Kit and republishes them through AUTOSAR Sender-Receiver interfaces. Because its primary role is data forwarding, the component contains minimal processing logic, thereby reducing computational overhead.

The Algorithm_SWC was implemented as the computational core of the system. This component performs sensor processing, distance smoothing, relative-speed estimation, TTC calculation, FCW classification, and braking decision generation. The algorithm was divided into multiple runnables to improve readability, maintainability, and modularity.

The AEB_Actuator_SWC was implemented as an output management component. It receives throttle, brake, and warning commands from the Algorithm_SWC and forwards only updated values to the communication bus. This design minimizes unnecessary CAN traffic and improves overall communication efficiency.

The modular implementation strategy simplified testing and allowed individual components to be verified independently before complete system integration.

5.5 Testing and Validation Methodology

5.5.1 Validation Strategy

The primary objective of system validation was to verify that the implemented AEB system could correctly detect collision risks and respond appropriately under different driving conditions. Validation activities were performed throughout development rather than being postponed until the end of the project.

A layered validation strategy was adopted. Individual software modules were tested first, followed by subsystem validation and finally complete system validation. This approach enabled defects to be identified at an early stage, reducing debugging effort during later integration phases.

Validation focused on both functional and non-functional requirements. Functional validation assessed whether the system generated correct warnings and braking commands. Non-functional validation evaluated communication reliability, timing consistency, and overall system stability.

5.5.2 Unit Testing

Unit testing was performed on individual software modules before integration. Each runnable within the AUTOSAR Software Components was tested independently using controlled input values.

For example, TTC calculation functions were evaluated using known distance and velocity combinations to verify mathematical correctness. Distance smoothing algorithms were tested using noisy sensor inputs to ensure stable output behavior. Warning classification functions were verified by applying TTC values corresponding to each FCW threshold.

These tests confirmed that each algorithmic component behaved as expected before integration into the complete system.

5.5.3 Integration Testing

After unit testing, integration testing was conducted to verify communication between interconnected modules. Integration testing focused on validating data flow throughout the complete processing chain.

The first integration test verified communication between CARLA and the Python gateway. The second test validated TCP communication between the gateway and SIL Kit participants. Subsequent tests confirmed successful data exchange between SIL Kit and the AUTOSAR environment.

CANoe monitoring tools were used extensively during integration testing. Live signal observation enabled rapid identification of communication issues, message timing problems, and incorrect signal values.

The successful completion of integration testing demonstrated that all system components could operate together within a unified Software-in-the-Loop environment.

5.6 Scenario-Based Verification

5.6.1 Vehicle-to-Vehicle Scenarios

Vehicle-to-vehicle scenarios were designed to evaluate the AEB system's ability to prevent rear-end collisions. In these tests, the ego vehicle approached a lead vehicle under various speed and distance conditions.

The system continuously monitored obstacle distance and calculated TTC values. As the gap between vehicles decreased, the FCW logic generated progressively stronger warnings. When the TTC value fell below the emergency threshold, automatic braking commands were issued.

The observed behavior closely matched the expected response defined during system design, confirming the effectiveness of the implemented collision assessment logic.

5.6.2 Vulnerable Road User Scenarios

Additional validation scenarios involved pedestrians and cyclists crossing the vehicle's path. These scenarios represent common urban driving hazards and are frequently included in modern AEB evaluation protocols.

The LiDAR sensor successfully detected vulnerable road users entering the vehicle's trajectory. TTC values were calculated in real time, allowing the system to assess collision risk and initiate braking when necessary.

These tests demonstrated that the system was capable of responding not only to vehicles but also to dynamic road users that present significant safety risks in urban environments.

5.6.3 System Performance Evaluation

System performance was evaluated using several metrics, including warning generation accuracy, braking response timing, communication latency, and overall system stability. Repeated simulation runs showed consistent system behavior across identical scenarios.

The fixed-step simulation architecture ensured deterministic execution, while SIL Kit communication provided reliable message delivery between components.

No significant synchronization issues were observed during testing, indicating that the selected architecture was suitable for safety-critical applications requiring predictable timing behavior.

5.7 Quality Assurance Practices

Quality assurance activities were incorporated throughout the project lifecycle to improve software reliability and maintain development consistency.

Version control was managed using Git and GitHub, allowing team members to track modifications, review code changes, and maintain a complete development history.

Branching strategies were used to isolate experimental features before merging them into the primary codebase.

Code reviews were conducted periodically to identify implementation issues and ensure compliance with project requirements. Documentation was updated continuously to maintain consistency between system design and implementation.

In addition, simulation results were archived and compared across multiple development iterations. This process allowed regressions to be detected quickly and ensured that improvements did not unintentionally introduce new defects.

5.8 Chapter Summary

This chapter presented the methodology adopted for the development and validation of the proposed Autonomous Emergency Braking system. The chapter described the Agile-inspired development model, project planning activities, implementation workflow, testing procedures, and quality assurance practices used throughout the project lifecycle.

The methodology emphasized incremental development, modular implementation, continuous verification, and progressive integration. Through systematic testing and scenario-based validation, the project successfully demonstrated the integration of CARLA simulation, AUTOSAR Software Components, SIL Kit communication middleware, and CANoe diagnostic tools within a unified Software-in-the-Loop environment.